

Національний технічний університет України
«Київський політехнічний інститут» ім. Ігоря Сікорського
Факультет прикладної математики
Кафедра системного програмування і спеціалізованих комп'ютерних систем

Лабораторна робота №2

з дисципліни «Розробка програмного забезпечення»

Виконав студент групи: КВ-34
Чижов Владислав

Київ 2026

1. Тема та мета роботи

Тема: Інтерфейси.

Мета: навчитися працювати з топіками, сервісами та діями в ROS 2; налаштовувати політики QoS, реалізовувати «ледачу» підписку за допомогою matched_callback та ознайомитися з механізмом адаптації типів.

2. Використані засоби

Windows, Docker Desktop, ROS 2 Jazzy, C++, Visual Studio Code, CMake, colcon, rclcpp, rclcpp_action, std_msgs, rosidl.

3. Хід виконання роботи

3.1. Середовище та збирання пакетів

Для роботи використано ROS 2 Jazzy у Docker-контейнері, щоб усі необхідні бібліотеки були в одному середовищі. У робочому просторі ros2_ws створено два пакети: lab2_interfaces для власних типів сервісу й дії та lab2_comm для програм на C++. Проєкт збирався командою colcon build --symlink-install, а після збирання підключалося середовище install/setup.bash. У результаті обидва пакети вдалося зібрати, і ROS 2 розпізнав їх за назвами. Такий поділ виявився зручним, бо інтерфейси можна використовувати в різних вузлах.

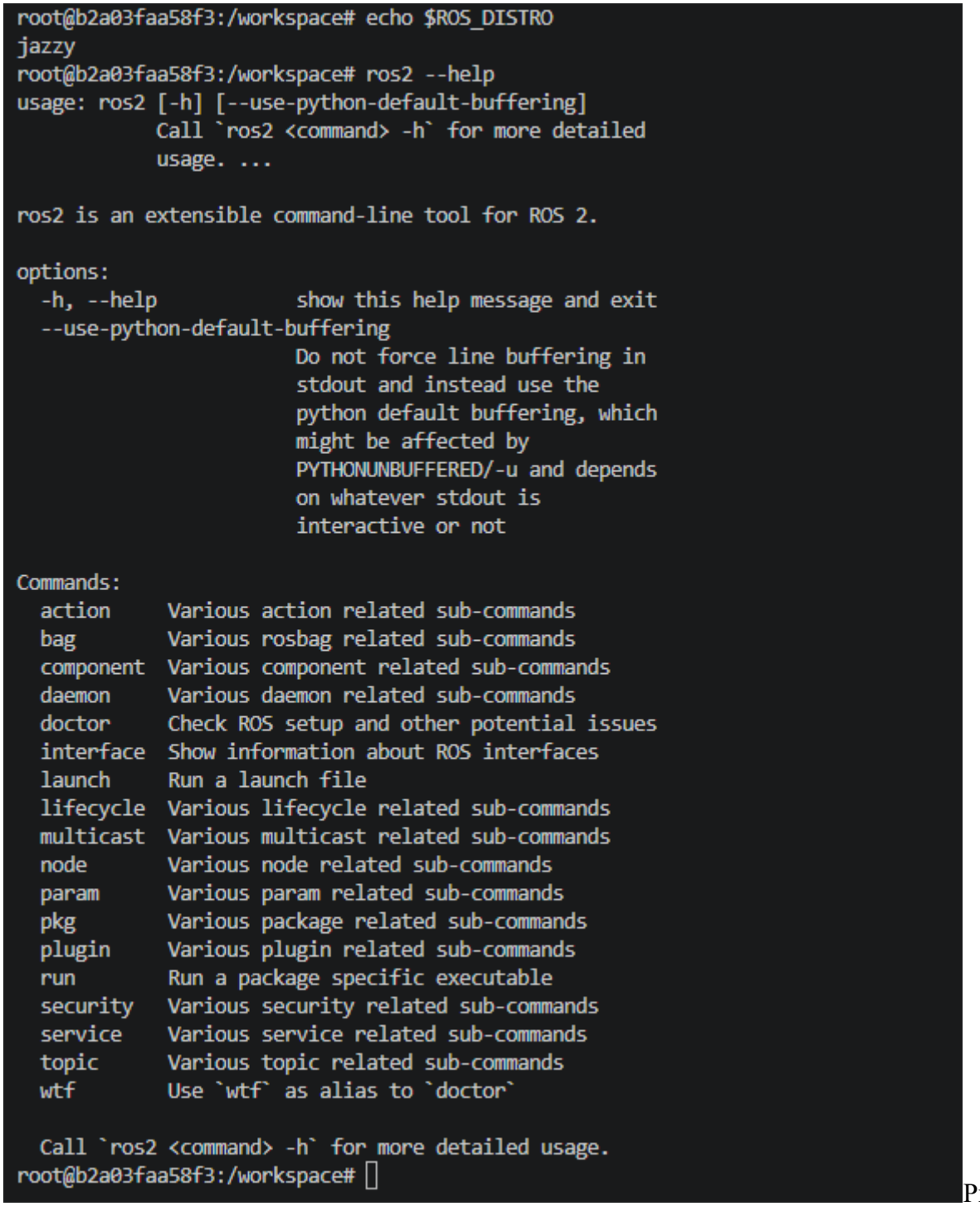


Рисунок 1 – Перевірка ROS 2 у Docker-контейнері

```
root@b2a03faa58f3:/workspace/ros2_ws# colcon build --symlink-install
Starting >>> lab2_interfaces
Finished <<< lab2_interfaces [6.53s]
Starting >>> lab2_comm
Finished <<< lab2_comm [2.22s]

Summary: 2 packages finished [9.07s]
root@b2a03faa58f3:/workspace/ros2_ws# source install/setup.bash
root@b2a03faa58f3:/workspace/ros2_ws# ros2 pkg list | grep lab2
lab2_comm
lab2_interfaces
root@b2a03faa58f3:/workspace/ros2_ws#
```

Рисунок 2 – Успішне збирання пакетів lab2_interfaces та lab2_comm

3.2. Topics та Lazy Subscription

Створено вузол data_filter, який отримує повідомлення std_msgs/msg/String з теми /input_topic, переводить латинські літери у верхній регістр і публікує результат у /uppercase_topic. Роботу перевірено на коротких тестових рядках: повідомлення надсилалися через ros2 topic pub, а результат переглядався через ros2 topic echo. Окремо використано matched_callback для відстеження підписників вихідної теми. Якщо слухачів немає, вхідна підписка вимикається, тому вузол не витрачає ресурси на непотрібну обробку. Після підключення слухача підписка створюється знову.

```
root@b2a03faa58f3:/workspace# ros2 topic pub --once /input_topic std_msgs/msg/String "{data: 'hello ros2'}"
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='hello ros2')

root@b2a03faa58f3:/workspace# ros2 topic pub --once /input_topic std_msgs/msg/String "{data: 'laboratorna robota'}"
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='laboratorna robota')

root@b2a03faa58f3:/workspace#
```

```
PS D:\kpi\rpz\lab2> docker exec -it ros2_lab2 bash
root@b2a03faa58f3:/workspace# source /opt/ros/jazzy/setup.bash
root@b2a03faa58f3:/workspace# source /workspace/ros2_ws/install/setup.bash
root@b2a03faa58f3:/workspace# ros2 topic echo /uppercase_topic'^C
root@b2a03faa58f3:/workspace# ros2 topic echo /uppercase_topic std_msgs/msg/String --qos-reliability reliable --qos-durability transient_local
data: HELLO ROS2
---
data: LABORATORNA ROBOTA
---
```

Рисунок 3 – Переведення тексту у верхній регістр

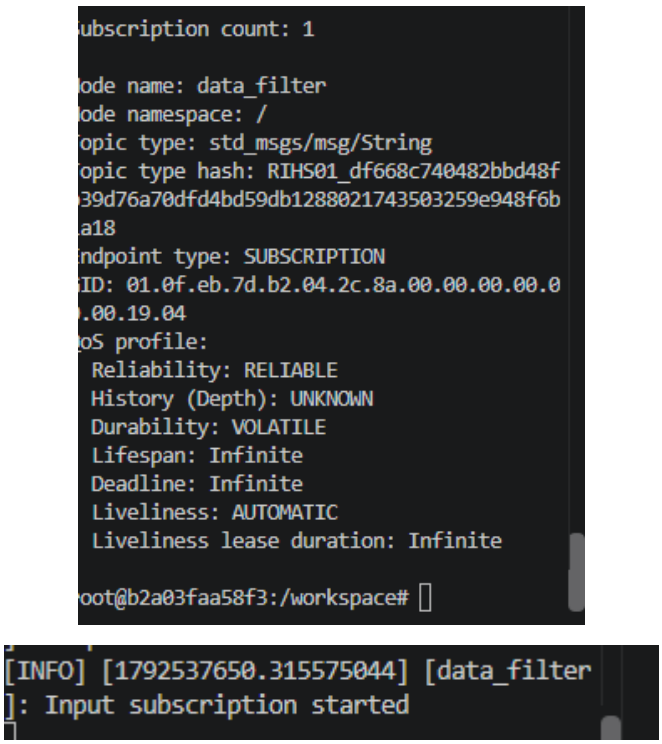


Рисунок 4 – Увімкнення і вимкнення підписки при зміні кількості слухачів

3.3. Налаштування QoS

Для вихідного видавця налаштовано профіль KeepLast(1), Reliable та Transient Local. Завдяки цьому видавець зберігає одне останнє повідомлення, яке може отримати новий сумісний підписник. Перевірка повторного підключення показала, як працює збереження останніх даних. У програмі також передбачено QosOverridingOptions для параметрів Depth, Reliability і Durability. Я зрозумів, що QoS впливає не лише на надійність, а й на сумісність між видавцем та підписником. При цьому після зміни C++ коду потрібна повторна компіляція, навіть якщо збірку виконано з --symlink-install.

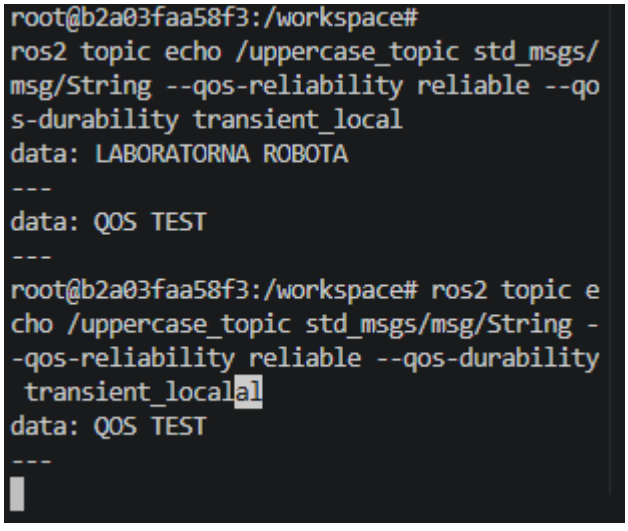


Рисунок 5 – Перевірка параметрів QoS вихідної теми

3.4. Асинхронний сервіс та інтроспекція

Для обміну у форматі «запит — відповідь» створено інтерфейс ResetFilter.srv і сервіс /reset_filter. Клієнт виконує асинхронний запит через async_send_request та очікує відповідь за допомогою future.wait_for з обмеженням часу. Під час перевірки сервер прийняв запит і повернув повідомлення про успішне виконання. Сам фільтр у цьому навчальному прикладі не зберігає стан, тому «скидання» демонструє саме механізм сервісу. Також було налаштовано інтроспекцію через приховану тему _service_event. Однак під час перевірки з CLI команда з параметром --include-hidden-topics завершилася помилкою, тому отримання подій інтроспекції на цьому скриншоті не підтверджене.

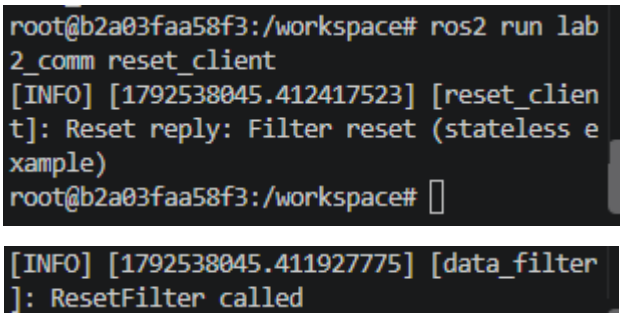


Рисунок 6 – Відповідь на асинхронний виклик сервісу ResetFilter

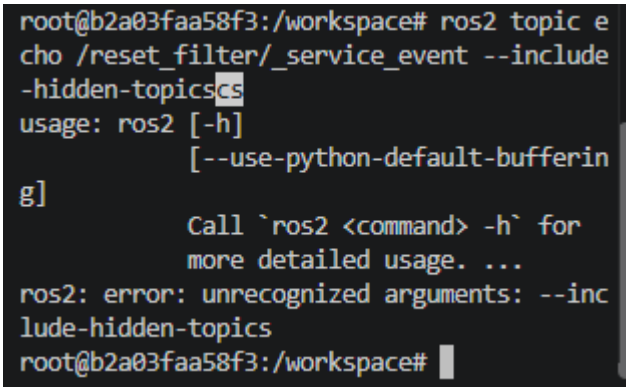


Рисунок 7 – Помилка CLI під час перевірки інтроспекції сервісу

3.5. Action-сервер

Для довшої операції реалізовано Action-сервер ProcessArray. Він приймає масив цілих чисел та обробляє їх по одному в окремому потоці, поступово підраховуючи суму. Після кожного елемента надсилається feedback з кількістю вже оброблених чисел, тому можна бачити хід виконання без очікування кінцевої відповіді. Для масиву [1, 2, 3, 4, 5] отримано результат 15. У коді також передбачено обробку скасування цілі. На практиці стало зрозуміло, чому для тривалих операцій Action зручніший за звичайний сервіс.

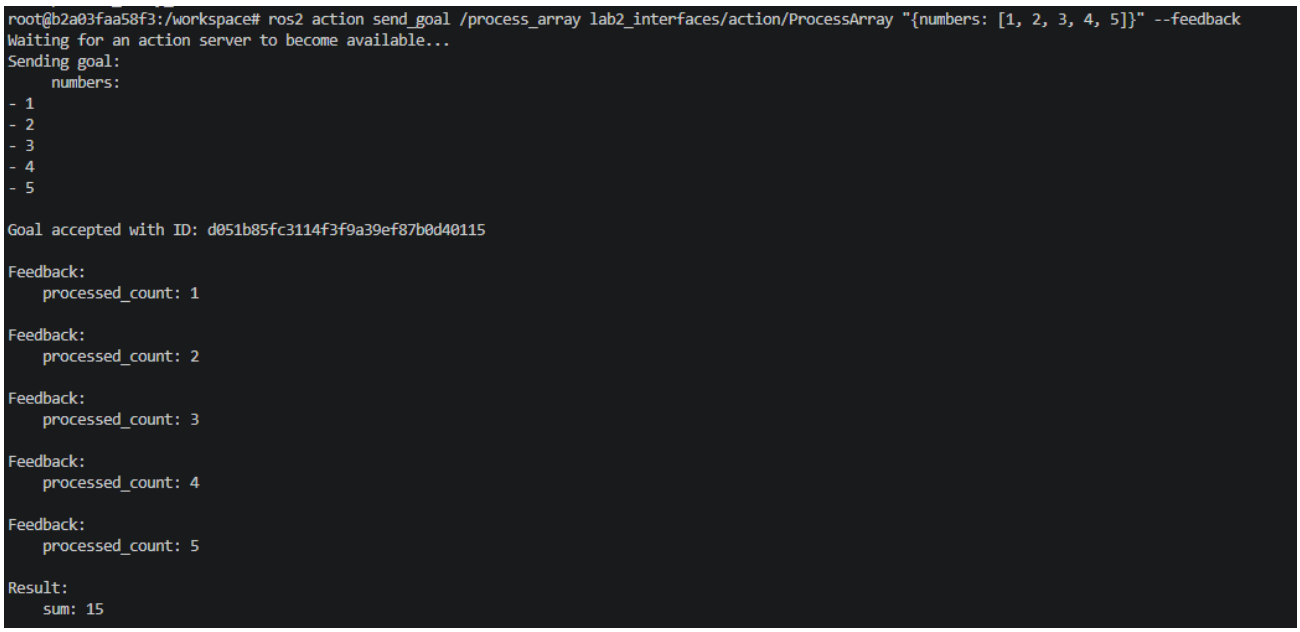


Рисунок 8 – Feedback та результат роботи Action-сервера

4. Висновки

У цій лабораторній роботі я краще розібрався, як вузли ROS 2 передають дані між собою. Спочатку налаштував робочий простір і збирання двох пакетів, а потім перевірив надсилання повідомлень через теми. Найбільше часу пішло на роботу з підписками: для Lazy Subscription важливо, щоб слухач вихідної теми справді був підключений. Після цього стало зрозуміліше, навіщо потрібен matched_callback. Також перевірив налаштування QoS, асинхронний виклик сервісу та виконання Action із проміжними повідомленнями feedback. Під час перевірки інтроспекції виникла проблема з параметром CLI, тому цей момент ще потребує уточнення. Загалом робота допомогла побачити різницю між темами, сервісами й діями та зрозуміти, у яких ситуаціях кожен з цих механізмів корисний.

6. Посилання на вихідний код

Git-репозиторій: https://github.com/mehnn/ROS2_lab2